TurtleBot kinematics

A TurtleBot is a small differential-drive robot: two driven wheels on one axle and caster balls that keep it level. In this assignment you derive how such a robot moves, answer short questions about the model, and test it against a physics simulation.

What you need to know. The Background section below gives everything the questions use. You do not need any kinematics beyond it. Lynch and Park, *Modern Robotics*, covers the same material in more depth. Read it if a question is unclear:

- Chapter 2: section 2.1 (degrees of freedom), section 2.3 (configuration space) and section 2.4 (velocity constraints). You can download chapter 2 as a PDF.
- Chapter 3: section 3.1 (rigid-body motions in the plane). You can download chapter 3 as a PDF.
- Chapter 13: subsection 13.3.1.2 (the differential-drive robot) and section 13.4 (odometry). You can download chapter 13 as a PDF.
- For Question 4: the section from velocity to motion in motion representations, where a pose is a matrix in SE(2) and a twist is a matrix in se(2).

How it works. Questions 1 to 5 are written. Question 4 is a short calculation, and the others ask for two or three sentences each. Questions 6 to 8 ask for a few lines of code, followed by a check cell that prints `passed`, and one sentence about what you see. Cells marked **Provided** are complete; run them without editing. If you restart the runtime, run every cell from the top again.

Question	Topic	Points
1 to 5	Written questions	50
6	The kinematic model against the simulation	15 (code 10, sentence 5)
7	The body twist of a turning robot	15 (code 10, sentence 5)
8	Odometry	20 (code 15, sentence 5)

Setup

Provided. The simulation runs on a CPU, so the default Colab runtime is enough. On it, the whole notebook takes about ten minutes, almost all of it simulation. If you have a GPU runtime (Runtime > Change runtime type), the simulation uses it and runs much faster. The first cell installs the simulation libraries.

```
import importlib
import importlib.util
import site
import subprocess
import sys

if importlib.util.find_spec("newton") is None or importlib.util.find_spec("reru
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "newtc
    # rerun-sdk is found through a .pth file, which Python reads only at startu
    for path in site.getsitepackages():
        site.addsitedir(path)
    importlib.invalidate_caches()
```

```
import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import warp as wp

wp.config.log_level = wp.LOG_WARNING
import newton
import newton.viewer

RECORDINGS = Path("recordings")
RECORDINGS.mkdir(exist_ok=True)
print("simulation device:", wp.get_device())
```

```
simulation device: cuda:0
```

The simulated TurtleBot

Provided. The robot has the geometry of a TurtleBot3 Waffle, the robot of the class repository. Its two numbers are below: in your code, write `WHEEL_RADIUS` for r and `HALF_SEPARATION` for d. Each wheel's motor tracks a commanded angular speed. The simulation knows nothing about the kinematic model: the chassis moves only through friction at the wheel contacts.

```
WHEEL_RADIUS = 0.033      # r, in meters
HALF_SEPARATION = 0.1435  # d, half the distance between the two wheels, in met
```

The next cell builds the simulation. You do not need to read it. `simulate(...)` runs at 60 frames per second and returns the frame times, the chassis configurations (ϕ, x, y) and the wheel angles (θ_L, θ_R). `test_course` is the drive used in Questions 6 and 8:

straight for 4 s at $u_L = u_R = 5$ rad/s, then along an arc for 4 s at $u_L = 3$, $u_R = 5$ rad/s.

Provided simulation (run it; you do not need to read it)

Show code

```
simulation ready
```

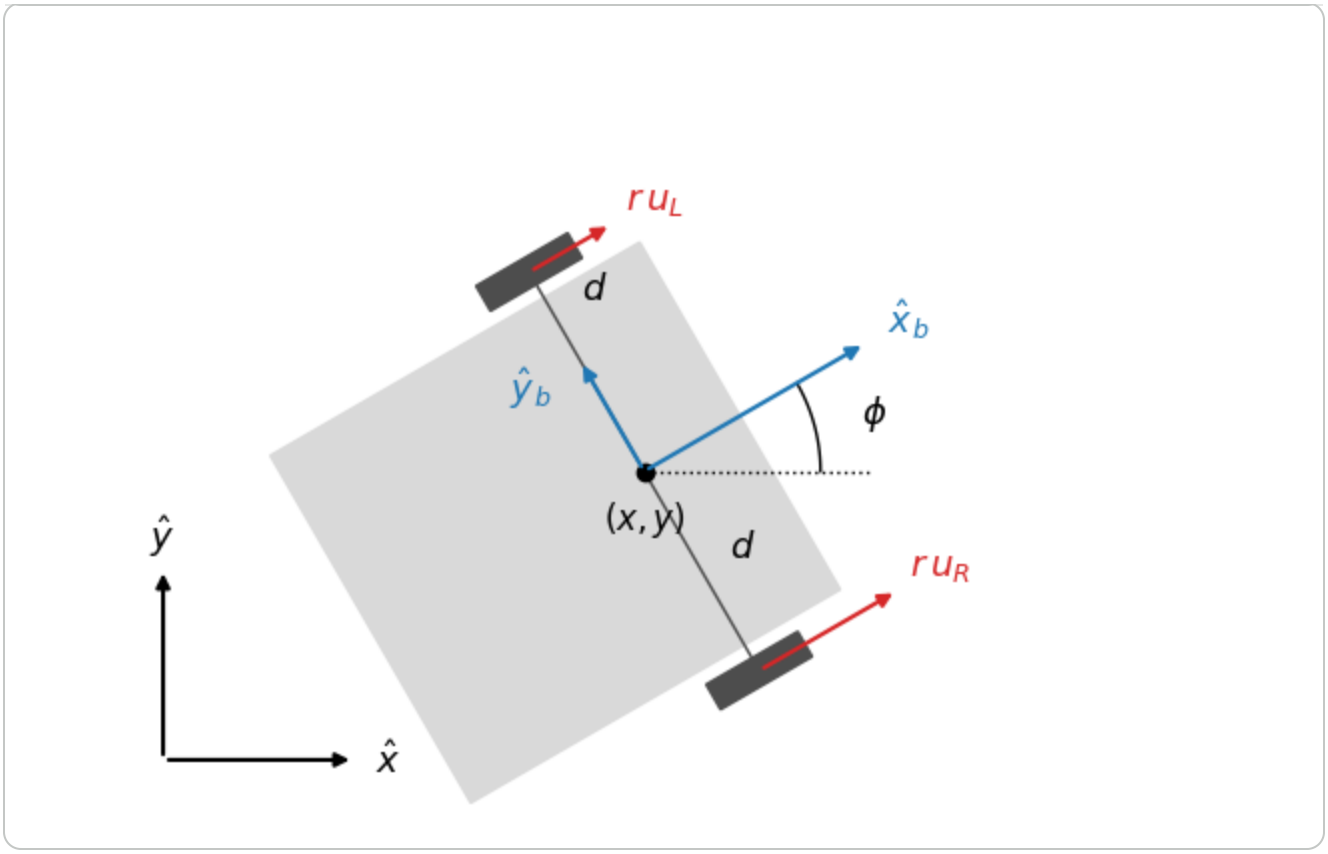
Background: how a differential drive moves

Configuration. Fix a world frame on the floor, with axes $\hat{x}$ and $\hat{y}$. The configuration of the chassis is $q = (\phi, x, y)$. The point (x, y) is the midpoint of the axle, and the heading ϕ is the angle from $\hat{x}$ to the direction the robot faces, counterclockwise positive.

Body frame. Attach a second frame to the chassis at the midpoint of the axle. Its axis $\hat{x}_b$ points forward along the heading, and $\hat{y}_b$ points to the robot's left. The left wheel sits a distance d to the left of the midpoint and the right wheel a distance d to the right. Both wheels have radius r.

Provided figure

Show code



Top view of the chassis at configuration (ϕ, x, y). The red arrows are the forward speeds of the two wheel centers, drawn for a robot turning left.

The wheels roll without slipping. A wheel that turns at angular speed u (in rad/s) moves its center forward at speed ru. It cannot slide sideways along its axle.

From wheel speeds to chassis motion. Describe the motion of the chassis by its forward speed v along $\hat{x}_b$ and its turning rate $\omega = \dot{\phi}$, counterclockwise positive. A point of the chassis a distance y_b to the left of the midpoint moves forward at $v - \omega y_b$: when the robot turns left, points on its left side slow down and points on its right side speed up. The left wheel is at $y_b = d$ and the right wheel at $y_b = -d$, so

$$ru_L = v - \omega d, \qquad ru_R = v + \omega d.$$

Adding and subtracting these two equations gives

$$v = \frac{r\,(u_L + u_R)}{2}, \qquad \omega = \frac{r\,(u_R - u_L)}{2d}. \tag{1}$$

Equal wheel speeds drive the robot straight. Unequal speeds turn it toward the slower wheel.

No sideways motion. Neither wheel center can move along $\hat{y}_b$, so the midpoint between them cannot either. The velocity of the axle midpoint always points along the heading.

Body twist. The three rates of the chassis along the body axes form the *body twist* $\mathcal{V}_b = (\omega, v_x, v_y)$: the turning rate, the forward velocity and the sideways velocity (section 3.1 of Lynch and Park). They are measured in a fixed frame that lines up with the body frame at that instant, not relative to the moving body frame, which would see the chassis at rest. For a differential drive, $v_x = v$ and $v_y = 0$.

Odometry. A robot cannot measure its configuration directly. Odometry estimates it from the wheel angles alone: it computes how far each wheel turned since the last reading and adds up the resulting motions step by step (section 13.4).

Part A. Written questions (50 points)

Answer each question in the cell below it: two or three sentences, or for Question 4 the calculation and one or two sentences.

Question 1 (10 points)

How many degrees of freedom does the TurtleBot's chassis have on a flat floor? What is its configuration space, and why is the heading a point on a circle S^1 rather than on the real line?

Hint: count the numbers you need to place the chassis on the floor; the Background lists them. For the heading, ask what happens to the robot when ϕ increases by 2π.

In the book: section 2.1 shows that a coin lying on a table has three degrees of freedom (chapter 2 PDF, pages 2 to 5). Section 2.3.1 gives the C-space of a planar rigid body, such as the chassis of a mobile robot (page 15).

The chassis has 3 degrees of freedom: placing it on the floor takes three numbers, its position (x, y) and its heading ϕ . Its configuration space is $\mathbb{R}^2 \times S^1$: $\mathbb{R}^2$ for the planar position (x, y) and S^1 for the heading ϕ. The heading lies on a circle because increasing ϕ by 2π leaves the robot in the same orientation, so ϕ and ϕ + 2π are the same configuration.

Question 2 (10 points)

The wheels cannot slide sideways. Write this as a constraint $A(q)\dot{q} = 0$ with $q = (\phi, x, y)$. Why does it restrict the robot's velocities but not the configurations it can reach?

Hint: the robot's sideways direction is $\hat{y}_b = (-\sin\phi, \cos\phi)$. The sideways velocity is the dot product of $(\dot{x}, \dot{y})$ with $\hat{y}_b$, and it must be zero. For the second part, think about how a driver parks a car in a space along the curb.

In the book: the rolling coin of section 2.4 has the same kind of constraint (Figure 2.11 and equations 2.9 and 2.10, chapter 2 PDF, page 21). The next page explains why such a constraint does not reduce the configurations the coin can reach (page 22).

The wheels can't slide sideways, so −ẋ sin ϕ + ẏ cos ϕ = 0, i.e. A(q)q̇ = 0 with A(q) = [0, −sin ϕ, cos ϕ]. No function g(ϕ, x, y) has these as its derivatives, because the allowed direction changes with the heading, so the constraint is nonintegrable, i.e. nonholonomic. It limits the allowed velocities but not the reachable configurations. Like the rolling coin, the robot can reach any pose by combining forward motion and turning, as in parallel parking.

Question 3 (10 points)

Invert equation (1): give the wheel speeds u_L and u_R that produce a forward speed v and turning rate ω. Which wheel speeds make the robot turn in place? The axle midpoint moves on a circle of signed radius $\rho = v/\omega$, positive when the robot turns left. Write ρ in terms of u_L and u_R.

Hint: the two wheel equations in the Background, $ru_L = v - \omega d$ and $ru_R = v + \omega d$, already relate each wheel speed to v and ω. For the radius, divide v by ω from equation (1).

In the book: subsection 13.3.1.2 gives the model as equation 13.15 (chapter 13 PDF, page 11). The text right after equation 13.18 gives the wheel speeds for a given (v, ω) (page 14).

Solving r·u_L = v − ωd and r·u_R = v + ωd gives u_L = (v − ωd)/r and u_R = (v + ωd)/r. The robot turns in place (v = 0) when the wheels spin at equal and opposite speeds, u_L = −u_R. Dividing v by ω from equation (1) gives the radius ρ = v/ω = d(u_R + u_L)/(u_R − u_L), positive (a left turn) when u_R > u_L.If u_L = u_R, the robot goes straight (ρ → ∞), and if u_L = −u_R, ρ = 0 and the robot spins in place .

Question 4 (10 points)

A TurtleBot starts at $x_0 = 2$ m, $y_0 = 1$ m with heading $\phi_0 = \pi$, so it faces the negative $\hat{x}$ direction. Its pose is the $SE(2)$ matrix

$$T_0 = \begin{bmatrix} \cos\phi_0 & -\sin\phi_0 & x_0 \\ \sin\phi_0 & \cos\phi_0 & y_0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For $t = 2$ s it drives with the constant forward speed $v = 0.5$ m/s and turning rate $\omega = -\pi/4$ rad/s, and no sideways velocity. Written as a matrix in $\mathfrak{se}(2)$, its body twist is

$$\hat{\xi} = \begin{bmatrix} 0 & -\omega & v \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For a constant body twist, the pose after time t is $T(t) = T_0 \exp(\hat{\xi} t)$. With $\Delta\phi = \omega t$, the exponential has the closed form

$$\exp(\hat{\xi} t) = \begin{bmatrix} \cos\Delta\phi & -\sin\Delta\phi & \dfrac{v}{\omega}\sin\Delta\phi \\ \sin\Delta\phi & \cos\Delta\phi & \dfrac{v}{\omega}(1-\cos\Delta\phi) \\ 0 & 0 & 1 \end{bmatrix}.$$

Compute $\exp(\hat{\xi} t)$ and $T(t)$, and read off the final configuration (ϕ, x, y). Then answer in one or two sentences: did the robot turn left or right, what path did the axle midpoint follow, and why is the product written $T_0 \exp(\hat{\xi} t)$ and not $\exp(\hat{\xi} t)\, T_0$?

Hint: you only need two numbers for $\exp(\hat{\xi} t)$: $\Delta\phi$ and v/ω. In $T(t)$, the last column holds the new position, and the first column is $(\cos\phi, \sin\phi)$ of the new heading.

In the course: the section from velocity to motion in motion representations writes a pose as a matrix in $SE(2)$ and a twist as a matrix in $\mathfrak{se}(2)$.

Given. $x_0 = 2$, $y_0 = 1$, $\phi_0 = \pi$, $v = 0.5$, $\omega = -\pi/4$, $t = 2$.

The parameters of the exponential

$\Delta\phi = \omega t = (-\pi/4)(2) = -\pi/2$

$v/\omega = 0.5 / (-\pi/4) = -2/\pi$

Since $\cos(-\pi/2) = 0$ and $\sin(-\pi/2) = -1$:

```
(v/ω) sin Δϕ       = (−2/π)(−1)     =  2/π
(v/ω)(1 − cos Δϕ)  = (−2/π)(1 − 0)  = −2/π
```

Substituting into the closed form,

```
                ⎡ cos Δϕ   −sin Δϕ   (v/ω) sin Δϕ      ⎤   ⎡  0    1    2/π ⎤
exp(ξ̂t) =      ⎢ sin Δϕ    cos Δϕ   (v/ω)(1 − cos Δϕ) ⎥ = ⎢ −1    0   −2/π ⎥
                ⎣   0         0          1            ⎦   ⎣  0    0    1   ⎦
```

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Since $\cos \phi_0 = \cos \pi = -1$ and $\sin \phi_0 = \sin \pi = 0$,

$$T_0 = \begin{bmatrix} -1 & 0 & 2 \\ 0 & -1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$T(t) = T_0 \exp(\xi t)$. Writing (i, j) for row i and column j:

$$\begin{aligned}
(1,1) &= (-1)(0) + (0)(-1) + (2)(0) &&= 0 \\
(1,2) &= (-1)(1) + (0)(0) + (2)(0) &&= -1 \\
(1,3) &= (-1)(2/\pi) + (0)(-2/\pi) + (2)(1) &&= 2 - 2/\pi \\
(2,1) &= (0)(0) + (-1)(-1) + (1)(0) &&= 1 \\
(2,2) &= (0)(1) + (-1)(0) + (1)(0) &&= 0 \\
(2,3) &= (0)(2/\pi) + (-1)(-2/\pi) + (1)(1) &&= 1 + 2/\pi \\
(3,j) &= (0, 0, 1)
\end{aligned}$$

Therefore

$$T(t) = \begin{bmatrix} 0 & -1 & 2 - 2/\pi \\ 1 & 0 & 1 + 2/\pi \\ 0 & 0 & 1 \end{bmatrix}$$

The upper-left block is a rotation matrix with $\cos \phi = 0$ and $\sin \phi = 1$, so $\phi = \pi/2$. The last column holds the position. Hence

$(\phi, x, y) = (\pi/2, 2 - 2/\pi, 1 + 2/\pi) \approx (1.571, 1.363, 1.637)$

Check. $\phi = \phi_0 + \Delta\phi = \pi - \pi/2 = \pi/2$.

Since $\omega < 0$, the robot turned right (clockwise), and the axle midpoint followed a circular arc of radius $|v/\omega| = 2/\pi \approx 0.64$ m, a quarter circle of length $vt = 1$ m. The product is $T_0 \exp(\xi t)$, not $\exp(\xi t)\, T_0$, because the twist is a body twist, measured in the robot's own frame, so the motion is applied relative to the robot's current pose by multiplying on the right.

Question 5 (10 points)

Odometry estimates the robot's configuration from its wheel angles alone. Why does its error grow over time, and what do robots add to keep an accurate estimate?

Hint: odometry adds up one small motion per wheel reading. What happens to an error made in one step, when every later step starts from it?

In the book: the first paragraph of section 13.4 (chapter 13 PDF, page 34).

Odometry works by adding up small movements from the wheel readings, and each new step starts from the last estimate. So if the wheels slip or skid a little, or the integration is slightly off, that error stays in the estimate and every later step adds to it, which means the error keeps growing. To fix this, robots use other sensors like GPS, landmarks, or laser range finders, whose errors don't build up, and they combine them with odometry using something like a Kalman filter or a particle filter.

Part B. Experiments (50 points)

Question 6 (15 points)

The configuration rate of the robot is $\dot{q} = (\dot{\phi}, \dot{x}, \dot{y})$, with $\dot{x}$ and $\dot{y}$ measured in the world frame.

Task. Write `diff_drive_qdot(q, u_L, u_R)`, which returns $\dot{q}$ as an array of shape `(3,)`. Compute v and ω from the wheel speeds with equation (1). Then $\dot{\phi} = \omega$. The velocity of the axle midpoint has magnitude v and points along the heading ϕ: split it into its $\hat{x}$ and $\hat{y}$ components. The provided `integrate` applies your function with small time steps.

Hint: a vector of length v at angle ϕ from $\hat{x}$ has components $v\cos\phi$ along $\hat{x}$ and $v\sin\phi$ along $\hat{y}$. If a check fails, its message says which motion went wrong.

In the book: subsection 13.3.1.2 gives this model as equation 13.15 (chapter 13 PDF, page 11). Equation 13.18 writes the same motion with v and ω as the inputs (page 14).

```
def diff_drive_qdot(q, u_L, u_R):
    phi = q[0]
    v = WHEEL_RADIUS * (u_L + u_R) / 2      # equation (1), with WHEEL_RADIUS f
    omega =  WHEEL_RADIUS * (u_R - u_L) / (2 * HALF_SEPARATION)  # equation (1)
    phi_dot = omega
    x_dot = v * np.cos(phi)
    y_dot = v * np.sin(phi)
    return np.array([phi_dot, x_dot, y_dot])
```

```
def integrate(q0, u_L, u_R, duration, dt=0.01):
    path = [np.asarray(q0, float)]
    for _ in range(int(round(duration / dt))):
        path.append(path[-1] + dt * diff_drive_qdot(path[-1], u_L, u_R))
```

```
    return np.array(path)


straight = integrate([0.0, 0.0, 0.0], 5.0, 5.0, 1.0)
assert np.allclose(straight[-1], [0.0, 5.0 * WHEEL_RADIUS, 0.0]), "equal wheel
spin = integrate([0.0, 0.0, 0.0], -2.0, 2.0, 1.0)
assert np.allclose(spin[-1], [2.0 * WHEEL_RADIUS / HALF_SEPARATION, 0.0, 0.0]),
north = integrate([np.pi / 2, 0.0, 0.0], 5.0, 5.0, 1.0)
assert np.allclose(north[-1], [np.pi / 2, 0.0, 5.0 * WHEEL_RADIUS]), "facing +y
print("passed")
```

```
passed
```

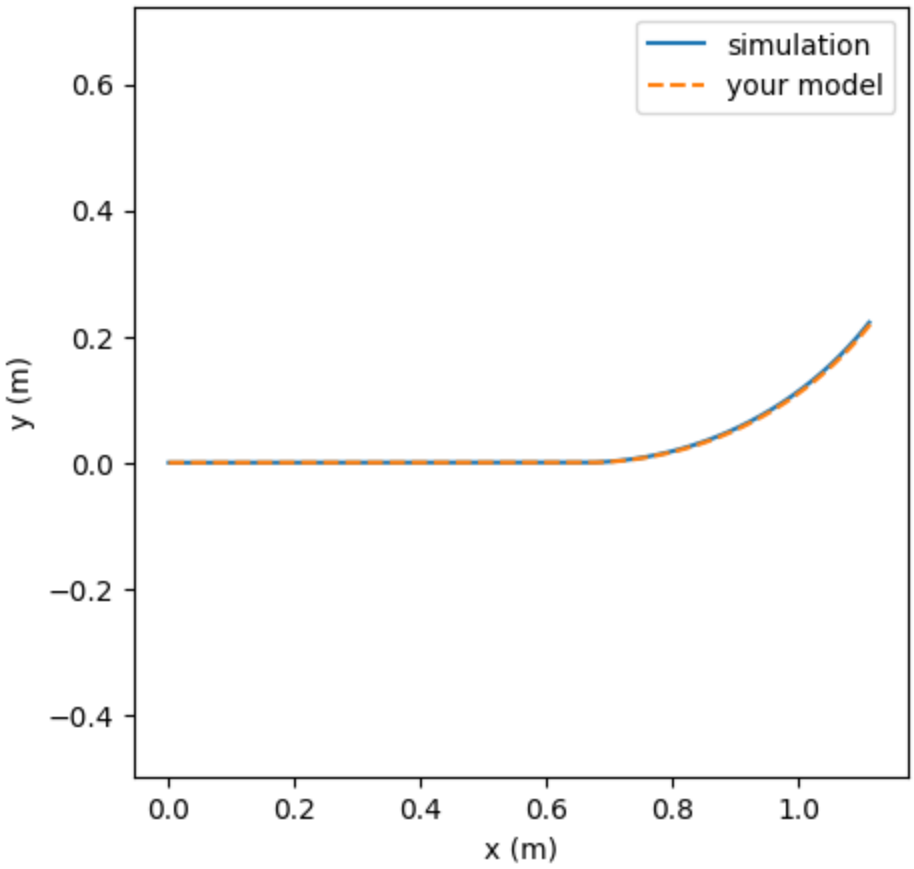
Provided. The simulated TurtleBot drives `test_course`: straight for 4 s at $u_L = u_R = 5$ rad/s, then along an arc for 4 s at $u_L = 3$, $u_R = 5$ rad/s. The cell runs your model from the same starting configuration and compares where the two end up. On a CPU runtime the first simulation takes about two minutes because the simulator compiles it first; later short drives take under a minute.

The cell also saves a recording of the drive as `recordings/turtlebot.rrd`. To watch it, download the file from the Files panel on the left of Colab and open it in the Rerun web viewer.

```
model, chassis, drives = build_turtlebot()
t_sim, pose_sim, angles_sim = simulate(model, chassis, drives, test_course, 8.0
leg1 = integrate(pose_sim[0], 5.0, 5.0, 4.0)
leg2 = integrate(leg1[-1], 3.0, 5.0, 4.0)
path_model = np.vstack([leg1, leg2[1:]])
gap = np.hypot(*(pose_sim[-1, 1:] - path_model[-1, 1:]))
heading_gap = abs(angle_diff(pose_sim[-1, 0], path_model[-1, 0]))
print(f"final position gap {gap:.3f} m, heading gap {heading_gap:.3f} rad")
assert gap < GAP_TOL and heading_gap < HEADING_TOL, "the model and the simulati
print("passed")
```

```
Rerun web viewer running at: http://127.0.0.1:9090/?url=rerun%2Bhttp%3A%2F%2F127
final position gap 0.008 m, heading gap 0.012 rad
passed
```

```
plt.figure(figsize=(5, 5))
plt.plot(pose_sim[:, 1], pose_sim[:, 2], label="simulation")
plt.plot(path_model[:, 1], path_model[:, 2], "--", label="your model")
plt.axis("equal")
plt.xlabel("x (m)")
plt.ylabel("y (m)")
plt.legend()
plt.show()
```



In one sentence (5 points): does the simulated robot follow your model, and what would make the two drift apart?

Hint: the model assumes the wheels roll without slipping. When could a real or simulated wheel break that assumption?

In the book: the first paragraph of section 13.4 lists what makes wheel motion and chassis motion disagree (chapter 13 PDF, page 34).

Answer. The simulated path closely follows the kinematic model, while wheel slip, motor response, or imperfect wheel dimensions can make the two paths drift apart.

Question 7 (15 points)

The simulation reports configurations in the world frame. To check the Background's claim that the robot never moves sideways, you need its velocity in the body frame.

Task. Write `world_to_body(phi, x_dot, y_dot)`. It returns (v_x, v_y), the components of the world-frame velocity $(\dot{x}, \dot{y})$ along the heading $\hat{x}_b = (\cos\phi, \sin\phi)$ and along the robot's left $\hat{y}_b = (-\sin\phi, \cos\phi)$. The inputs may be arrays, so use `np.cos` and `np.sin`.

Hint: the component of a vector along a unit vector is their dot product, so $v_x = \dot{x}\cos\phi + \dot{y}\sin\phi$. Write v_y the same way with $\hat{y}_b$. The two checks test a robot facing $+\hat{y}$ and a robot facing $+\hat{x}$; work them out by hand first.

In the book: section 3.1 writes the body axes $\hat{x}_b$ and $\hat{y}_b$ in fixed-frame coordinates (Figure 3.3 and equations 3.2 and 3.3, chapter 3 PDF, page 5) and introduces the planar twist (ω, v_x, v_y) (page 9).

Provided. The check cell drives the simulated robot along an arc at $u_L = 3$, $u_R = 5$ rad/s. It estimates $(\dot{\phi}, \dot{x}, \dot{y})$ from the simulated configurations, converts them to the body twist (ω, v_x, v_y) with your function, and compares the result with equation (1).

```
def world_to_body(phi, x_dot, y_dot):
    v_x = x_dot * np.cos(phi) + y_dot * np.sin(phi)
    v_y = -x_dot * np.sin(phi) + y_dot * np.cos(phi)
    return v_x, v_y
```

```
assert np.allclose(world_to_body(np.pi / 2, 0.0, 1.0), (1.0, 0.0)), "facing +y,
assert np.allclose(world_to_body(0.0, 0.0, 1.0), (0.0, 1.0)), "facing +x, movir

model, chassis, drives = build_turtlebot()
t_arc, pose_arc, _ = simulate(model, chassis, drives, lambda t: (3.0, 5.0), 4.0
# estimate the rates of phi, x and y from how much the pose changes between fra
# You don't need to fully understand these lines to answer the questions.
phi_arc = np.unwrap(pose_arc[:, 0])
omega_sim = np.gradient(phi_arc, t_arc)
v_x, v_y = world_to_body(phi_arc, np.gradient(pose_arc[:, 1], t_arc), np.gradie
steady = slice(len(t_arc) // 4, None)  # skip the first second, while the motor
v_model, omega_model = WHEEL_RADIUS * (3.0 + 5.0) / 2, WHEEL_RADIUS * (5.0 - 3.
print(f"turning rate:     model {omega_model:.3f} rad/s, simulation {omega_sim[
print(f"forward speed:    model {v_model:.3f} m/s, simulation {v_x[steady].mear
print(f"largest sideways speed in the simulation: {np.abs(v_y[steady]).max():.4
assert abs(omega_sim[steady].mean() - omega_model) < OMEGA_TOL, "measured and p
assert abs(v_x[steady].mean() - v_model) < V_TOL, "measured and predicted forwa
assert np.abs(v_y[steady]).max() < VBY_TOL, "the sideways body velocity should
print("passed")
```

```
turning rate:     model 0.230 rad/s, simulation 0.227 rad/s
forward speed:    model 0.132 m/s, simulation 0.131 m/s
largest sideways speed in the simulation: 0.0003 m/s
passed
```

In one sentence (5 points): which statement in the Background does the near-zero sideways velocity confirm, and what physical fact makes it true?

Hint: look back at the constraint you wrote in Question 2.

In the book: the no-slip constraint of the rolling coin in section 2.4 (chapter 2 PDF, page 21).

Answer. The near-zero sideways velocity confirms the Background's no-sideways-motion constraint, which is true because the wheels cannot slide along their axle.

Question 8 (20 points)

Between two wheel-angle readings, the wheels turn by $\Delta\theta = (\Delta\theta_L, \Delta\theta_R)$. Equation (1) with wheel angles in place of wheel speeds gives the motion of the chassis over that step as a body twist (equation 13.34):

$$\mathcal{V}_b = F\,\Delta\theta, \qquad F = r\begin{bmatrix} -1/(2d) & 1/(2d) \\ 1/2 & 1/2 \\ 0 & 0 \end{bmatrix}.$$

Provided. `displace(q, twist)` returns the configuration reached from `q` by following one constant body twist for one step, as in section 13.4.

Task. Write `odometry(q0, wheel_angles)`. `wheel_angles` has one row (θ_L, θ_R) per reading. Start at `q0`. For each pair of consecutive readings, compute $\mathcal{V}_b = F\Delta\theta$ and apply `displace`. Return all configurations, including `q0`, as an array of shape `(N, 3)`.

Hint: `np.diff(wheel_angles, axis=0)` gives every $\Delta\theta$ at once, and `F @ delta` is the matrix product. Append each new configuration to `path`, and apply `displace` to the last one.

In the book: section 13.4 gives F for a differential drive as equation 13.34 (chapter 13 PDF, page 35). `displace` computes equations 13.35 and 13.36 (page 36).

```
def displace(q, twist):
    # move configuration q = (phi, x, y) by a constant body twist (omega, v_x,
    # You don't need to fully understand this function to answer the questions.
    w, vx, vy = twist
    if abs(w) < 1e-12:
        dx_b, dy_b = vx, vy
    else:
        dx_b = (vx * np.sin(w) + vy * (np.cos(w) - 1)) / w
        dy_b = (vy * np.sin(w) + vx * (1 - np.cos(w))) / w
    c, s = np.cos(q[0]), np.sin(q[0])
    return np.asarray(q, float) + np.array([w, c * dx_b - s * dy_b, s * dx_b +
```

```
def odometry(q0, wheel_angles):
    F = WHEEL_RADIUS * np.array([[-1 / (2 * HALF_SEPARATION), 1 / (2 * HALF_SEP
    path = [np.asarray(q0, float)]
```

```
    for delta in np.diff(wheel_angles, axis=0):
        twist = F @ delta
        path.append(displace(path[-1], twist))
    return np.array(path)
```

Provided. The check cell drives `test_course` again and runs your odometry on the simulated wheel angles.

```
est = odometry([0.0, 0.0, 0.0], np.column_stack([np.linspace(0, 10, 11), np.lir
assert est.shape == (11, 3) and np.allclose(est[-1], [0.0, 10 * WHEEL_RADIUS, 0
model, chassis, drives = build_turtlebot()
_, pose_course, angles_course = simulate(model, chassis, drives, test_course, 8
est_course = odometry(pose_course[0], angles_course)
gap_odo = np.hypot(*(est_course[-1, 1:] - pose_course[-1, 1:]))
print(f"odometry ends {gap_odo:.3f} m from the true final position")
assert gap_odo < ODO_TOL, "odometry should end close to the true position on th
print("passed")
```

```
odometry ends 0.004 m from the true final position
passed
```

Provided. On an 8 s drive, odometry stays within a few millimeters of the truth. The next cell drives a winding course for 60 s, alternating 10 s arcs to the left and to the right, and runs your odometry on it. The plots show both paths and the position error of odometry over time. On a CPU runtime the drive takes about five minutes.

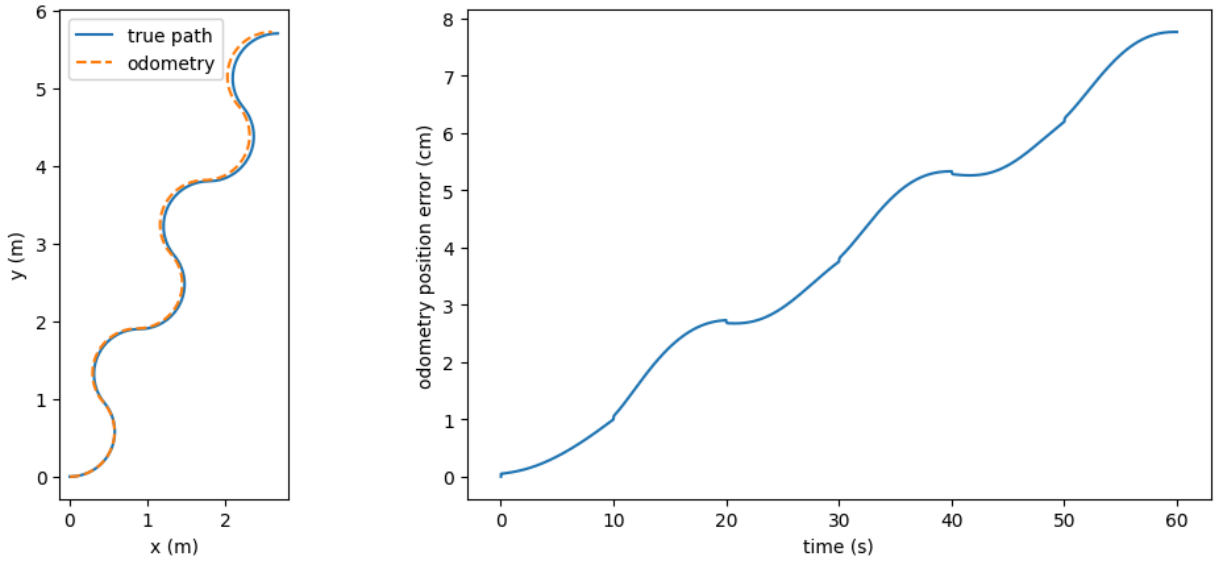
```
def winding_course(t):
    # wheel speeds (u_L, u_R) in rad/s: 10 s arcs, alternately to the left and
    return (3.0, 5.0) if (t % 20.0) < 10.0 else (5.0, 3.0)


model, chassis, drives = build_turtlebot()
t_long, pose_long, angles_long = simulate(model, chassis, drives, winding_cours
est_long = odometry(pose_long[0], angles_long)
error_long = np.hypot(est_long[:, 1] - pose_long[:, 1], est_long[:, 2] - pose_l
for seconds in (8, 20, 40, 60):
    print(f"odometry error after {seconds:2d} s: {100 * error_long[seconds * 60
```

```
odometry error after  8 s: 0.7 cm
odometry error after 20 s: 2.7 cm
odometry error after 40 s: 5.3 cm
odometry error after 60 s: 7.8 cm
```

```
fig, (ax_path, ax_error) = plt.subplots(1, 2, figsize=(11, 4.5))
ax_path.plot(pose_long[:, 1], pose_long[:, 2], label="true path")
ax_path.plot(est_long[:, 1], est_long[:, 2], "--", label="odometry")
```

```
ax_path.set_aspect("equal")
ax_path.set_xlabel("x (m)")
ax_path.set_ylabel("y (m)")
ax_path.legend()
ax_error.plot(np.arange(1, len(error_long) + 1) / 60, 100 * error_long)
ax_error.set_xlabel("time (s)")
ax_error.set_ylabel("odometry position error (cm)")
plt.tight_layout()
plt.show()
```



In one sentence (5 points): how does the odometry error change over the 60 s drive, and